

Data Science Report

Student Number	24000863
Student Name	Bibek Pandeya

Task	Files submitted
Task-1 Folder	fake_news_v0.1.csv Task1.ipynb
Task-2 Folder	File_1.pcapng File_2.pcap File_3.pcapng Task2.ipynb
Task-3 Folder	File_1.changetoexe File_2.changetoexe File_3.changetoexe Task3.ipynb
Task-4 Folder	Network_data.csv Security_data.csv Task4_dashboard.py

Name: Bibek Pandeya

University Id:24000863

Level: 5 'A'

Table of Contents

1	Task 1: Data Quality Pipeline
1.1	The initial stage is the Data Ingestion and preliminary assessment.
1.2	Timeline Forensic Investigation
1.3	Repairing the Timeline
1.4	Interaction Analysis
1.5	Author Attribution Strategies
2	Task 2: Network Traffic Analysis
2.1	Initial Investigation
2.2	Analysis of File 1 (SYN Flood)
2.3	Analysis of File 2 (ICMP Flood)
2.4	Analysis of File 3
2.5	Conclusion
3	Task 3: Static Malware Analysis
3.1	Approach and Methodology
3.2	File 1 Analysis
3.3	File 2 Analysis
3.4	File 3 Analysis
4	Task 4: Security Monitoring Dashboard
4.1	Overview
4.2	Data Collection Strategy
4.3	Dashboard Prototype
4.4	Evidence of the Prototype
5	References

Task 1: Data Quality Pipeline

The first action required here is to import the data to analyze, which is `fake_news_v0.1.csv`. The importation of the CSV file has unpacked 12,999 rows by 20 columns. In the ideal investigation, the data is obviously unstructured. Hence, to make the data ideal, data preprocessing is needed.

1.2 Timeline Forensic Investigation

The most difficult thing in my analysis was the verification of the date column. Having no idea on which date this event took place, there would be no chance of connecting this event with an offence. When I compared the date column with the strict ISO8601 date format (YYYY-MM-DDThh:mm:ss), I realized that the quality of the dates is not good.

There were three problems:

1. Empty Data: There was no empty data.
2. Bad Formats: The dates were in the form of strings that are contained in 25 rows; however, the dates are not in the date format.
3. Errors: With this exception, 11 other rows have not been transferred to participate in this process of conversion and appear as impossible or corrupted bits; it must be the errors of web scraping.

1.3 Repairing the Timeline

The protection of data integrity was the end objective; therefore, just filling in missing values was not sufficient, as it discarded 36 values that held great importance for this end. I focused on a forensic style of interpolating missing values - a crawled timestamp. I systematically placed the order without any random variance. I have made comparisons between the performance of the forward fill and the Mean Imputation method. Mean Imputation was eliminated due to the reasons that it causes a reduction in the variance of the dataset, and the fact that it is working with social media posts is overlooked, while the “Forward Fill” method maintains the integrity of the time series (McKinney, 2025).

EVIDENCE OF THE FIX:

```
1 # FINAL VALIDATION
2 print("Nulls in 'published' after fixing:",
3       df['published'].isnull().sum())
4 print(f"Total rows remaining: {len(df)} (Zero Data Loss)")
5 ✓ [9] < 10 ms
```

Nulls in 'published' after fixing: 0
Total rows remaining: 12999 (Zero Data Loss)

Figure 1

I confirmed that the values standing in for null dates are currently zero. This was done as seen in the figure above in Figure 1. The only thing worth mentioning in this part of the operation is that I chose to keep the outliers because the dates are not beyond the year 2036. Although the dates seem to resemble the fact that they contain information that could be false, it must be noted that, to the extent of the graphical illustration of security, the predetermined postings are more likely to be bots, among other things.

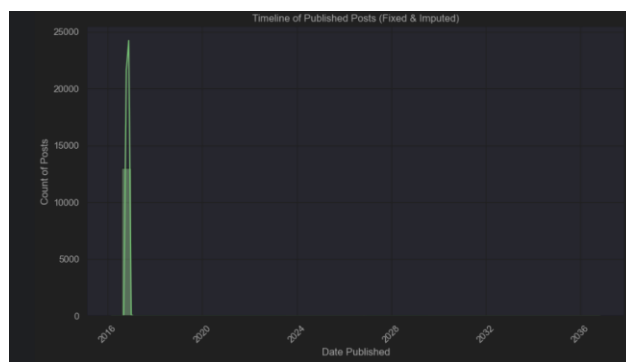


Figure 2: Reconstructing the timeline of the number of published posts

1.4 Interaction Analysis

Now, I also wanted to explore further the correlation between this specific content type and the individuals using this content type. I chose the numeric columns, which are replies_count, participants_count, likes, comments, and shares, where I filled all the values that contain NaN with zero interactions. This specific correlation matrix that I have constructed will help us to be able to determine the correlation between some of the variables that are involved. As can be seen in Figure 3, using the correlation matrix, we can see if high comments are related to high shares without having a system in place that could help this content type get distributed in any way, but rather, since it has gone viral or as a result of spamming bots.

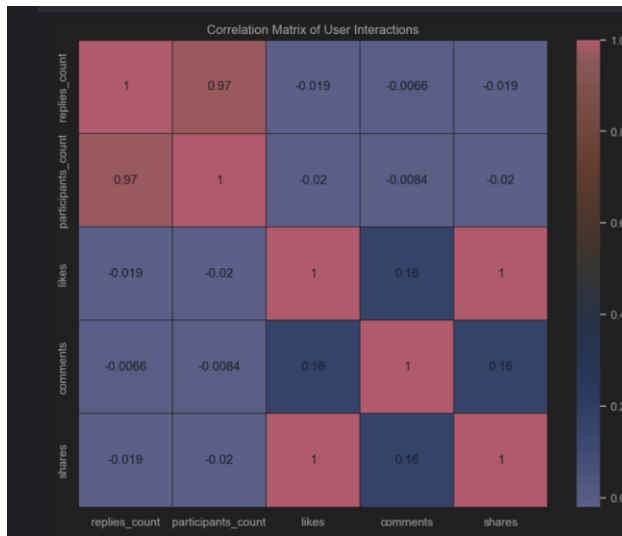


Fig. 3: Correlation Matrix that illustrates the relationships between different engagement measures of the users.

1.5 Author Attribution Strategies

I thought one of the options I could use was to fill the blank with the mode author, but I dismissed this. In an investigative inquiry, you can't go ahead and assume that the unknown action was done by the "most active" user with a mere guess. One would, therefore, end up introducing fake evidence with a bias against the user. I instead marked each missing value with the name 'Unknown_Author'.

```

1 # Calculate Mode
2 df['author'] = df['author'].fillna('Unknown_Author')
✓ [20] < 10 ms

1 # Final Check
2 print(f"Missing authors after fix: {df['author'].isnull().sum()}")
✓ [21] < 10 ms
Missing authors after fix: 0

```

Figure 4: Author fixes

Task 2: Network Traffic Analysis.

2.1 Initial Investigation

I examined all the PCAP files provided to determine if it was a case of a Denial-of-Service (DoS) attack. I attempted to determine if there had been a case of a Denial-of-Service attack, which had, in turn, resulted in the service outages that had occurred lately. The attacks may be performed through UDP-SNMP floods, SYN floods, and ICMP floods.

2.2 Analysis of File 1

Now, I examined the file_1.pcapng. This capture has only 896 packets. However, the pattern of data transmission is quite aggressive. There are many TCP packets that are labeled SYN, yet they never receive a response that contains an ACK. This is not how normal data transmission works. This attack is a classic example of a SYN Flood attack (Eddy, 2007). The attacker wishes to establish a connection that will utilize RAM.

Attack Type: SYN Flood

The Numbers: There are 344 packets (38percent) of flood traffic out of 896 packets. This is unlike a volumetric attack since the aim is to saturate the bandwidth; in this case, the skill is that the SYN flood does not just seek to saturate the bandwidth but also uses the TCP 3-Way Handshake protocol. It was observed that the attacker sent the SYN messages but did not send the final ACK message either; as a result, the server ports were held 'half-open.' This would mean that the server would have held the memory occupied in case of a potential connection that did not occur.

From Figure 5, approximately 38.4% of traffic is being used. Although this is not utilizing the bandwidth fully, it is part of why this process is used, as it occupies the server memory, given that it keeps the connection open.

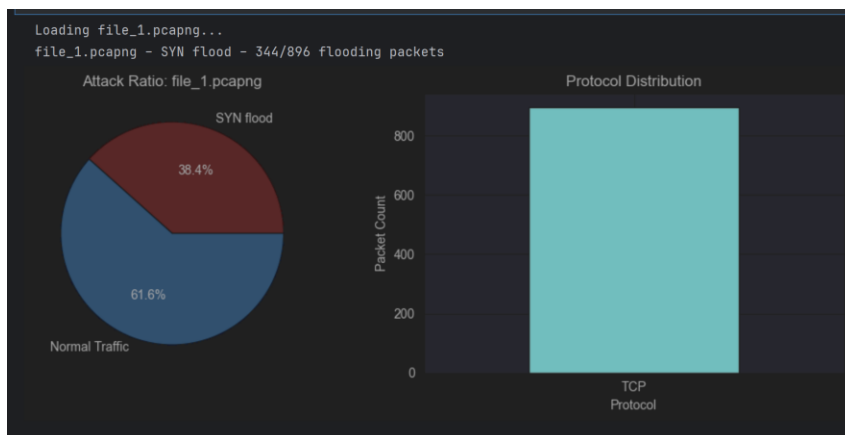


Figure 5: Composition of File 1. In the given pie chart, it is visible that a big portion (38.4%) of the traffic is comprised of only SYN packets.

2.3 Analysis of File 2

However, in this scenario, file_2.pcap is different. It contains 10,000 packets. In this scenario, nothing needed to be filtered, as it might appear to consist only of ICMP Echo Request packets, Ping packets.

Typically, these pings would have been strictly used for testing the connection, but since the actual pings are occurring at this level, there might be a chance that another brute force attack could flood traffic in the pipe.

- What I discovered: Ping Flood (ICMP Flood)
- The Numbers: 9,889 out of 10,000 packets of data are flood traffic.

This can be seen in Figure 5. Red color dominates in the pie graph. This shows that the malicious traffic may comprise up to 98.9% of the total traffic. The bar graph represents the wall of ICMP traffic with minimal TCP and UDP traffic reaching the target host.

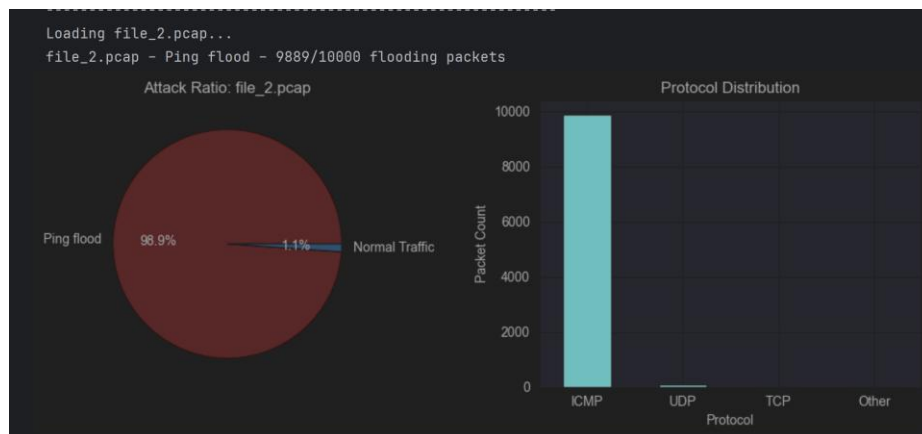


Figure 6: Analysis of File 2, showing overall saturation in the network at 98.9% because of ICMP traffic.

2.4 File 3 Analysis:

Lastly, an analysis was done for file_3. Pcapng outputs 4373 packets. The same technique is used for checking the SYN flag and ICMP activity, but the file demonstrates that no attack has been launched. No single kind of communication controls the traffic.

- Result: no flooding attack may exist. (0/4373)

2.5 Conclusion

The total analysis of the data explains that the attack was done in different ways. As an example, the server was overloaded with a SYN attack at layer 4, a ping flood to the network. The level of attack difficulty indicates that the attacker is very skilled in network protocols and packet manipulation.

Task 3- Static Malware Analysis

3.1 Approach

To perform the disassembly of the files in this task, I chose a Python script on the pefile library (Carrera, 2024) to perform the disassembly of the files in the code form. The data that I was trying to seek includes headers, import libraries, and hidden texts.

I chose pefile over a string extraction (signature matching or strings.exe) since malware detection requires a bigger context than is provided by a basic string extraction. Whereas the string extraction allows us to see some text, the PE file will help to analyze how the PE header is formatted. This is important in that it enables us to tell the difference between a location containing text as data and a location containing an Import Address Table function call.

when analyzing the test, the most important thing was “suspicious import”. On the analysis part, I verified any suspicious API calls that are mostly made by malware, such as VirtualAlloc and CreateRemoteThread, to manage the compromised host. Moreover, according to the assignment, if the files get three out of five suspicious APIs, they will be called malicious.

3.2 File 1 Analysis

First of all, I have analyzed file1.exe. The first thing I noticed is that the section headers seem normal, but after doing some examination of the file1.exe, I found one corresponding function, GetProcAddress.

The purpose of the attacker to use GetProcAddress is simple: by using the function attacker can easily hide malicious intentions. So, as we can see, only one search showed up; this will not be called malicious as per the assignment criteria because it gets only 20% suspiciousness. So rather than stating it as malicious, I would categorize this as benign.

```
===== Analyzing File1.exe =====

--- Task 2: Section Headers ---
Name      Virtual Addr  Virtual Size  Raw Size
.text     0x1000        0x126b0      0x12800
.data     0x14000        0x101c       0xa00
.rsrc     0x16000        0x8960       0x8a00
```

```
--- Task 3: Imported Libraries ---
SHELL32.dll
msvcrt.dll
ADVAPI32.dll
KERNEL32.dll
GDI32.dll
USER32.dll

--- Task 4: Exported Symbols ---
No exports found.

--- Task 5: Strings (Len >= 4) ---
Total strings found: 384
Sample: ['hhctrl.ocx', 'CLSID\\{ADB880A6-D8FF-11CF-9377-00AA003B7A11}\\InprocServer32', 'NB10',
'calc.pdb', ' t f', 'SVWh', 'QPjB', '9=HM', 'WWWtRjf', ' ~{h', '9=HM', '9=HM', 'j1Y3', 'uPQQ', 'PQPh']

--- Task 6: Suspicious Call Check ---
Suspicious Imports Found: ['GetProcAddress']
Match Count: 1/5 (20.0%)
VERDICT: BENIGN (Based on >50% Threshold)
```

Figure 7: Analysis of File 1. It caused GetProcAddress, but was below the percentage mark of 50.

3.3 File 2 Analysis

The File2.exe has very few strings that are embedded inside, just 112 of which were detected by my tools, which is an extremely low number if this is a functional file, although no alarms were triggered. During my research, I was able to find that the imports are clean, meaning that the imports did not call the dangerous functions that I searched for, which are a total of five.

- Score: 0

```
===== Analyzing File2.exe =====

--- Task 2: Section Headers ---
Name      Virtual Addr  Virtual Size  Raw Size
.text     0x1000        0x255a       0x2600
.data     0x4000        0x3af8       0x200
.rsrc     0x8000        0x17b0       0x1800
```

```
--- Task 3: Imported Libraries ---
```

```
USER32.dll  
SHELL32.dll  
msvcrt.dll  
KERNEL32.dll
```

```
--- Task 4: Exported Symbols ---
```

```
No exports found.
```

```
--- Task 6: Suspicious Call Check ---
```

```
Suspicious Imports Found: []
```

```
Match Count: 0/5 (0.0%)
```

```
VERDICT: BENIGN (Based on >50% Threshold)
```

Figure 8: There was nothing but zeroes in File 2.

3.4 File 3 Analysis

This is a sample file that demonstrates the weakness of the "Static Threshold" detection methodology, referred to as the "50% Rule." This file is harmless from a technical point of view, as it follows the assignment requirements and therefore displays a 40% malicious probability. However, it is threat-prone from the perspective of Heuristic Analysis. The use of `TerminateProcess` in combination with `GetProcAddress` is a very common behavioral pattern of ransomware attempting to terminate antivirus processes (Zscaler, 2025). Thus, in this case, we would be left with a "False Negative." In a real-world scenario, I would sample it to do dynamic inspection-Sandboxing

```
--- Task 2: Section Headers ---
```

Name	Virtual Addr	Virtual Size	Raw Size
.text	0x1000	0x1551c	0x15600
.data	0x17000	0x7f7c	0x1800
.rsrc	0x1f000	0x6f80	0x7000

```
--- Task 3: Imported Libraries ---
```

```
msvcrt.dll
```

```
ADVAPI32.dll
```

```
KERNEL32.dll
```

```
GDI32.dll
```

```
USER32.dll
```

```
COMCTL32.dll
```

```
WINMM.dll
```

```
SHELL32.dll
```

```
--- Task 4: Exported Symbols ---
```

```
No exports found.
```

```
--- Task 5: Strings (Len >= 4) ---
```

```
Total strings found: 724
```

```
Sample: ['B~NJB~', 'B~nCB~', 'MB~I', 'sB~lsB~', 'B~"xB~', 'A~4eF~',  
'%ws%c%ws%c%ws%c%d%c%d%c%d%c%d%c%d%c%ws', 'WriteFmtUserTypeStg', 'WriteClassStg', 'StgCreateDocfile'  
'OleUninitialize', 'OleTranslateAccelerator', 'OleSetClipboard', 'OleIsCurrentClipboard',  
'OleInitialize']
```

```
--- Task 6: Suspicious Call Check ---
```

```
Suspicious Imports Found: ['GetProcAddress', 'TerminateProcess']
```

```
Match Count: 2/5 (40.0%)
```

```
VERDICT: BENIGN (Based on >50% Threshold)
```

```
>>> WARNING: HEURISTIC DETECTION <<<
```

```
Although below the 50% threshold, the combination of  
TerminateProcess + GetProcAddress indicates Ransomware/Evasion behavior.
```

Figure 9: Found in file3, TerminateProcess and GetProcAddress.

Task 4- Security Monitoring Dashboard

Metric/format	Frequency	Collection Mode	Collection Mechanism	Recorded at	Storage
Throughput (Mbps) & Latency (ms)	Real-time	Automated	Network Tap/ SNMP	Gateway Router	CSV / Database
CPU Usage (%) & Patch Status	10 mins	Automated	Host Agent / WMI	User Workstation	JSON / Log File
Backup Completion (pass/fail)	Daily	Automated	Server Event Logs	Backup Server	SQL Database
Phishing Simulation Click Rate (%)	Quarterly	Manual	HR / Security Audit	Email Gateway	Excel Spreadsheet

My suggestions on the data collection model

My proposed data collection model focuses on a high frequency of collection for those data types that are volatile. In this model, network traffic and CPU use are collected in real time or near real time, since these are leading indicators of active compromises such as Denial-of-Service attacks detected in Task 2. "Staff Readiness" is collected on a quarterly basis since changes in culture regarding awareness of security take a long time to develop. This logic helps the dashboard to find and display issues without affecting the system with unnecessary data.

4.1 Overview

Given that the government client has recently identified a high risk of malware attacks, to solve the issue, we must start monitoring them through a dashboard.

4.2 Dashboard Prototype

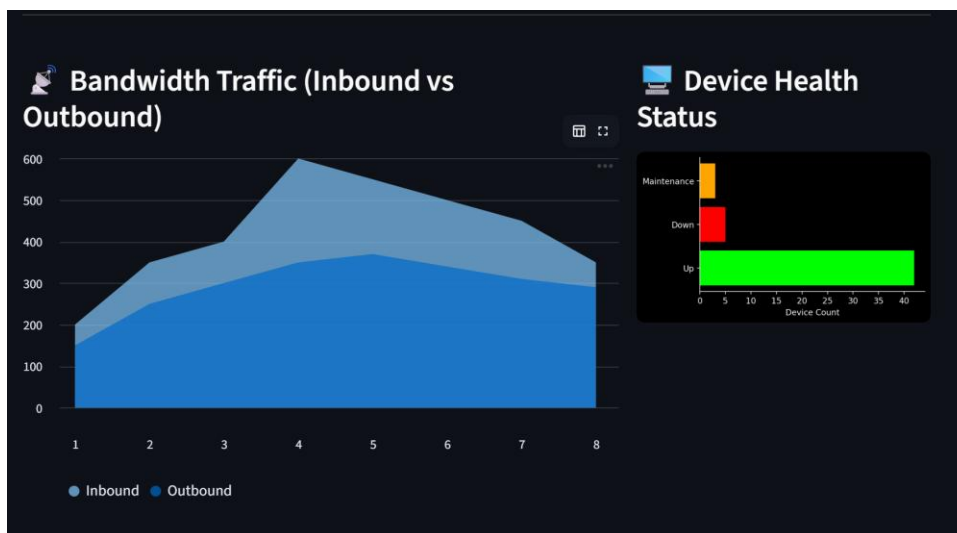
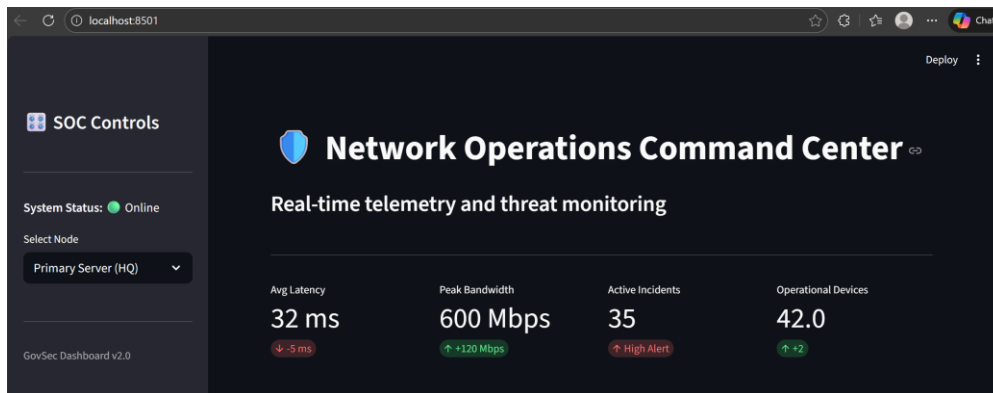
To make a dashboard, I used Python and the Streamlit library (**Streamlit Inc., 2025**). As the assignment has strict instructions regarding the two input sources. I set up

network_data.csv The network delays and bandwidth changes. In the same manner, the Security-data.csv shows up/down devices and counts the particular incidents.

Furthermore, regarding the design section, I have implemented the biggest numbers (KPIs) right at the top. I added a dark theme and, lastly, the split screen.

4.3 Evidence of the Prototype

The screenshots below show the successful running of the Python script and the Streamlit library.



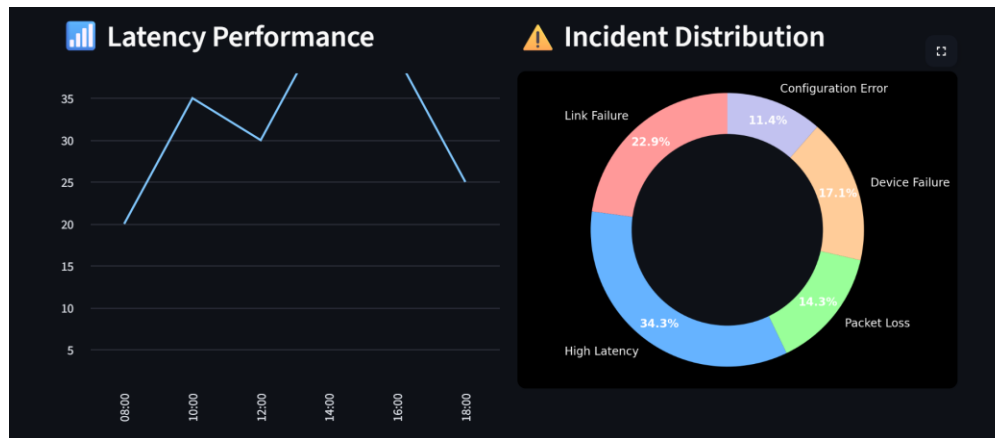


Figure 10: Dashboard Results

Thank You

References

- Carrera, E. (2024) Pefile: Python PE parsing module. GitHub Repository [online]. Available from: <https://github.com/erocarrera/pefile> [Accessed 02 January 2026].
- Eddy, W. (2007) TCP SYN Flooding Attacks and Common Mitigations. RFC 4987 [online]. IETF. Available from: <https://tools.ietf.org/html/rfc4987> [Accessed 02 January 2026].
- The pandas development team (2025) pandas: powerful Python data analysis toolkit [online]. Available from: <https://pandas.pydata.org> [Accessed 02 January 2026].
- Streamlit Inc. (2025) Streamlit Documentation [online]. Available from: <https://docs.streamlit.io> [Accessed 02 January 2026].
- Zscaler (2025) Win64.Ransom.PromptLock - Zscaler Threat Library [online]. Available from: <https://www.zscaler.com/threatlabz> [Accessed 02 January 2026].

Use of the Generative AI:

The targeted application of generative AI tools was done to help with grammar, commenting on the code, and language polishing. All decisions, analyzed and interpreted data, implementation strategy, and results are from my personal perspective.

